VAULT — Online Banking Web Application

Assignment 2 Report

Ziaul Haque Rafi | Student Number: A00096932
EEN1037 Web Application Development

Dublin City University

---

1. Introduction

VAULT is a fully functional web-based banking application I built using the Django framework. The idea was to create a platform where users can open accounts, move money around, and apply for loans — all through a clean browser interface. This project was developed for the Web Application Development assignment, and it gave me a chance to bring together everything from backend logic to front-end presentation in one cohesive system.

2. Objectives

Going into this project, I had a few clear goals in mind:

- Build a secure, production-minded web application from the ground up
- Implement the core features you'd expect from a real banking platform
- Create a seamless connection between the frontend and Django's backend
- Ensure data stays accurate and protected, especially during financial operations

3. System Overview

At a high level, VAULT works the way most modern web apps do: a user interacts with a page, that request travels through Django's view layer, data is pulled from or written to the database via models, and the result is rendered back as an HTML template. It's a straightforward pipeline, but getting it right for banking-style operations required careful attention at each step.

Request flow: User → Frontend → Views → Models → Database

4. Technologies Used

The technology stack I chose reflects a balance of simplicity and practicality:

- Python — the core language powering the backend
- Django — the web framework handling routing, ORM, and templating
- HTML & CSS — for structuring and styling the user interface

- Bootstrap — to make the UI responsive without writing custom grid code
- PostgreSQL — a robust relational database for storing all application data
- Docker — used to containerize the app and keep the environment consistent

5. System Architecture

VAULT follows Django's Model-View-Template (MVT) pattern. This isn't just a convention I followed for the sake of it — it actually made the codebase much easier to reason about and maintain:

- Models define the database structure and act as the single source of truth for data
- Views contain the business logic — deciding what data to fetch and what to do with it
- Templates handle everything the user actually sees, keeping presentation separate from logic

6. Database Design

I designed four core models to represent the main entities in the system:

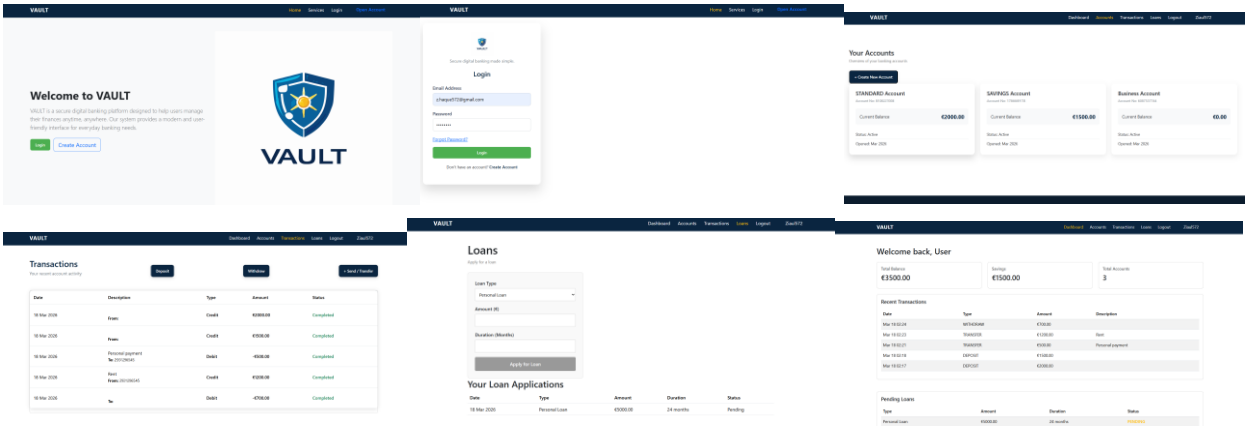
UserProfile: Extends Django's built-in user model with additional personal details.

BankAccount: Stores each account's details and tracks the running balance.

Transaction: Records every transfer, deposit, and withdrawal with a timestamp.

Loan: Captures loan applications along with their current approval status.

7. UI/UX



8. Features Implementation

Here's a breakdown of the main features I built and how they work:

Authentication: Users can register, log in, and log out. Sessions are managed securely throughout.

Accounts: Users can create bank accounts and view their details at any time.

Transactions: The app supports transfers between accounts, deposits, and withdrawals — all validated before processing.

Loans: Users can submit loan applications and check whether they've been approved or are still pending.

Profile: Users can update their personal information and upload a profile photo.

Dashboard: A central hub showing an at-a-glance view of all accounts and recent activity.

9. Security Considerations

Security wasn't an afterthought — I built it in from the start. A few of the key measures I put in place:

- Django's built-in authentication system handles login security and password hashing
- CSRF protection is active on all forms to prevent cross-site request forgery attacks
- Session management ensures users are properly authenticated throughout their visit
- All user inputs are validated before being processed or stored
- Financial operations use atomic transactions, so partial updates can't leave the database in an inconsistent state

10. Challenges Faced

This project pushed me in a few areas I hadn't worked in deeply before:

- Getting comfortable with Django's ORM took time, especially writing efficient queries for relational data
- Managing database migrations — particularly when models changed mid-development — required careful handling to avoid breaking the schema
- Debugging the transaction logic was tricky; ensuring that deposits and withdrawals updated balances correctly under all conditions took several iterations
- Session management had some unexpected edge cases, especially around login state after certain operations

11. Testing

I tested all major features manually, working through common user journeys like logging in, making a transfer, and applying for a loan. Beyond the happy path, I also tested edge cases that could trip up a banking app — things like trying to withdraw more than the available balance, submitting forms with invalid or missing data, and handling duplicate requests. These tests helped me catch and fix several issues before finalizing the submission.

12. Conclusion

Building VAULT was a genuinely rewarding experience. It brought together a lot of moving parts — database design, backend logic, front-end templating, and security — into something that actually works and looks like a real application. The project met all of the assignment objectives, and more importantly, it gave me a much stronger foundation in full-stack Django development that I'm excited to build on.